Demo video – what to do and what to say

Plant DDN-01 · 5-minute demo · step by step

> **How to use this guide.** Each step has a **DO** box (what you click or type) and a **SAY** box (what you say). Say it in your own words if that feels easier: the boxes are a guide, not something to read word for word. Speak slowly. Record each scene as a **separate clip** and join them at the end (Clipchamp: put the clips in order → Export → 1080p). You can cut out any waiting.

Before you record (do this once)

> **Copy tip:** every grey box can be copied as it is. Select the text in the box, press **Ctrl + C**, then in PowerShell press **Ctrl + V** (or right-click) and **Enter**. In the Neo4j browser, paste into the top box and press **Ctrl + Enter** (or the blue play button).

1. Update and run everything. Open PowerShell and copy:

```
cd "C:\Users\AKASH\OneDrive\Desktop\akash\Industrial-AI-Automobile-Manufacturing-Plant"
```

```
git checkout -- reports docs/ontology.md outputs
git pull
```

Check that `git pull` does **not** say "error" or "Aborting". Then:

```
.\run_all.bat
```

Wait for **"All 18 steps done"** and **"102 passed"**.

2. PowerShell window A (for the demo commands). Open a new PowerShell and copy:

```
cd "C:\Users\AKASH\OneDrive\Desktop\akash\Industrial-AI-Automobile-Manufacturing-Plant"
$env:PYTHONPATH = "src"
cls
```

3. PowerShell window B (Ask the plant). Open a new PowerShell and copy:

```
cd "C:\Users\AKASH\OneDrive\Desktop\akash\Industrial-AI-Automobile-Manufacturing-Plant"
.venv\Scripts\streamlit run apps\ask_app.py --server.port 8501
```

4. PowerShell window C (Review screen). Open a new PowerShell and copy:

```
cd "C:\Users\AKASH\OneDrive\Desktop\akash\Industrial-AI-Automobile-Manufacturing-Plant"
.venv\Scripts\streamlit run apps\review_app.py --server.port 8502
```

5. Browser tabs, in this order (copy each address into a new tab):

```
https://github.com/akashnegetive/Industrial-AI-Automobile-Manufacturing-Plant
```

```
http://localhost:7474
```

Neo4j login: user `neo4j`, password `ddn-plant-2026`.

```
http://localhost:8501
```

```
http://localhost:8502
```

6. Make things easy to read: PowerShell text bigger (Ctrl + mouse wheel), browser zoom 125% (Ctrl + plus), Windows "Do not disturb" on. **Never show your OpenAI key** on screen.

7. Recorder: OBS Studio with **Display Capture** and your microphone (or Clipchamp → Record → Screen). Do a 10-second test and play it back.

Scene 1 – Introduction (about 25 seconds)

DO: Show browser tab 1 (README, "Results" table).

SAY: "Hi, I'm Akash. This is my project for car plant DDN-01. In a car plant, one machine has many names. It has one name in SAP, another in the sensor system, another in the work orders and another on the drawings. My project connects all these names to one machine. It puts everything in a knowledge graph, and every link says where it came from and how sure we are. Everything runs with one command. I will show one robot signal and one car, from start to end."

DO: Move the mouse over the line "Tags F1 1.000, work orders F1 0.982".

SAY: "The matching is very accurate, and no wrong link was accepted automatically."

Scene 2 – One robot signal: from raw name to the graph (about 70 seconds)

DO: Go to PowerShell window A. Copy this, paste, press Enter:

```
.venv\Scripts\python -m ddn.demo tag DDN1_BIW_L2_R07_WELDCUR.PV
```

SAY: "This is one signal from the plant's sensor system. It is the weld current of a welding robot."

DO: Point at **block 1** (raw record).

SAY: "Here is the raw name. It says R07, but SAP calls this robot RB07. And the name does not say which station the robot is in."

DO: Point at **block 2** (parsed).

SAY: "First, my code splits the name: body shop, line 2, robot 7, weld current. It also finds the standard code for the measurement and its unit, kiloamps."

DO: Point at **block 3** (matched).

SAY: "Then the matcher finds the right machine: robot 7 in station 4 of line 2. It uses rules and a machine learning model. It is very sure, so the link is accepted automatically."

DO: Point at **block 4** (in the knowledge graph).

SAY: "Now the link is in the graph. The link also stores its source, the method, the confidence and the reason. The sensor values themselves stay in a file; the graph only keeps a pointer to them."

DO: Point at **block 5** (context).

SAY: "And now we can see the context: the robot belongs to station 4, line 2, the body shop and the plant. There was an anomaly on this signal, AN-003, and here is the work order that fixed it."

Scene 3 – The same signal as a picture in the graph (about 30 seconds)

DO: Go to browser tab 2 (Neo4j). Copy this query into the top box and press **Ctrl + Enter**. A small picture appears. If circles overlap, drag them with the mouse.

```
MATCH p1 = (t:TimeSeries {tag: 'DDN1_BIW_L2_R07_WELDCUR.PV'})
           <-[:hasTimeSeries]-(a:Asset)
MATCH p2 = (a)-[:partOf*1..5]->(:Site)
OPTIONAL MATCH p3 = (t)-[:hasEvent]->(:AnomalyEvent)
OPTIONAL MATCH p4 = (a)-[:hasEvent]->(:AnomalyEvent)
RETURN p1, p2, p3, p4
```

SAY: "This is the same signal in the graph. The signal hangs under robot 7. The robot is in station 4, the station is in line 2, then the body shop, then the plant. The anomaly is connected to the signal and to the robot."

DO: Click the line between the **signal** and the **robot**. Its details appear.

SAY: "When I click a link, I can see why the graph believes it: the method, the confidence and the status. So nobody has to guess where a link came from."

Scene 4 – One car from start to end (about 80 seconds)

DO: Go to PowerShell window A. Copy this, paste, press Enter:

```
cls
.venv\Scripts\python -m ddn.demo vin
```

SAY: "Now one car, from start to end. This car was built on the 8th of September."

DO: Point at **block 1** (route).

SAY: “Here is its route through the plant: 44 stations, from the body shop in the morning to the final check in the evening. Sometimes a reader misses the car. Then my code fills the gap from the line speed, and marks it as ‘inferred’.”

DO: Point at **blocks 2 and 3**.

SAY: “Here are the robots that worked on this car: welding, sealing and painting robots. And here are the torque results for its bolts. All are OK.”

DO: Point at **block 4** (parts and lots).

SAY: “Here are its parts, and the supplier batch of each part.”

DO: Point at **block 5** (defects).

SAY: “This is the important part. The car had a weld defect. The graph knows the cause: anomaly AN-003 on robot 7. That is the same robot as in my first example. And here is the work order that fixed the robot.”

Scene 4b – The same car as a picture (about 20 seconds, optional)

DO: Go to the Neo4j tab. Copy this query into the top box and press **Ctrl + Enter**:

```
MATCH p1 = (v:Vehicle {vin: 'MDDA5XK00D003924'})-[:hasDefect]->(:Defect)
            -[:causedBy]->(e:AnomalyEvent)<-[:hasEvent]-(:Asset)
MATCH p2 = (v)-[:containsPart]->(:Part {part_no: 'DOOR_SEAL'})
           -[:fromLot]->(:SupplierLot)
MATCH p3 = (v)-[:exposedTo]->(e)
RETURN p1, p2, p3
```

SAY: “And this is the same car as a picture: the car, its defect, the anomaly that caused it, and the robot where it happened. Here is also a part of the car with its supplier batch. A question that used to take a week of spreadsheets is now one query.”

Scene 5 – Ask the plant a question (about 60 seconds)

DO: Go to browser tab 3 (Ask the plant). Click the **Question** box, paste this question, press Enter and wait a few seconds:

```
Which robot caused the cold welds on 7 and 8 September, and which work order fixed it?
```

SAY (while it loads): “Here I can ask a question in normal English. An AI model turns my question into a graph query. My code checks the query first: it may only read, never change anything.”

SAY (when the answer appears): “Every sentence in the answer has its sources: the robot, the anomaly and the work order. If a sentence has no source, my code removes it. No source, no claim.”

DO: Click the expander **“Cypher (validated, run read-only)”**.

SAY: “Here is the query the AI wrote, so anyone can check it.”

DO: Paste this second question and press Enter:

```
Who was the shift supervisor on 9 September?
```

SAY: “And when the data does not have the answer, it says ‘I don’t know’. It does not guess. In my tests it never made up a fact.”

Scene 6 – A person checks a link, and the graph updates (about 60 seconds)

DO (6a): Go to the Neo4j tab. Copy this query into the top box and press **Ctrl + Enter**. It shows **no records**:

```
MATCH (a:Asset)-[r:hasWorkOrder]->(w:WorkOrder {wo_number: '4102964'})
RETURN a.fl AS asset, r.method AS method, r.confidence AS confidence,
       r.status AS status, r.evidence AS evidence
```

SAY: “This work order, number 4102964, is about the calibration of a tightening tool. The model was not sure enough about it, so it is not in the graph yet. Only links that are very sure, or checked by a person, go into the graph.”

DO (6b): Go to browser tab 4 (Review screen). Paste this number into the **search box** on the left, read the text, look at the proposed machine (nutrunner 14), then click **Approve**:

```
4102964
```

SAY: “Here an engineer checks the links the model is not sure about. The text says NR14, and the proposed machine is nutrunner 14. That is correct, so I approve it. The decision is saved with my name and the time.”

DO (6c): Go to PowerShell window A. Copy this, paste, press Enter:

```
.venv\Scripts\python -m ddn.graph --decisions
```

SAY: “Now I put the decision into the graph. It takes only a few seconds.”

DO (6d): Go back to the Neo4j tab. Run the **same query** again (Ctrl + Enter, or copy it again):

```
MATCH (a:Asset)-[r:hasWorkOrder]->(w:WorkOrder {wo_number: '4102964'})
RETURN a.fl AS asset, r.method AS method, r.confidence AS confidence,
       r.status AS status, r.evidence AS evidence
```

Now there is **one row**, with method **engineer**.

> **SAY:** "Same query, and now the link is there. It says 'engineer', with who approved it and when. The model also learns from this decision the next time it runs. So the graph always shows why it believes something: a rule, the model, or a person."

> **DO (optional):** Click the tab **"Backlog (data quality)"**.

> **SAY:** "Links that are very unlikely go here, to a backlog. For these, a second method looks at the graph and suggests the three most likely machines."

> **Before Scene 6:** run the 6a query once without recording. If it **already shows a row**, you approved this work order earlier. Then use **4102961** (robot 7's weld gun) instead: search for it in the review screen, and use this query in 6a and 6d. Approve only links you are sure about: decisions are real data.

```
MATCH (a:Asset)-[r:hasWorkOrder]->(w:WorkOrder {wo_number: '4102961'})
RETURN a.fl AS asset, r.method AS method, r.confidence AS confidence,
       r.status AS status, r.evidence AS evidence
```

Scene 7 – Closing (about 15 seconds)

> **DO:** Go back to browser tab 1 (README, "Results" table).

> **SAY:** "To sum up: the matching is very accurate, with no wrong links accepted automatically. All eight problems hidden in the sensor data were found. And the question box gives answers with sources, or says 'I don't know'. The limits are written in my report. Thank you for watching."

After recording

1. Watch the whole video once. Can you read the text? Is your voice clear? Is it about 5 minutes (up to 6 is fine)? Is your API key **not** visible?
2. If one scene went badly, record **only that scene** again.
3. Join the clips (Clipchamp → drag clips in order → Export → 1080p).
4. Upload to YouTube: studio.youtube.com → Create → Upload → Visibility **Unlisted** → copy the link.
5. Put the link in the README, or send it to me and I will add it.

All commands in one place

PowerShell window A

```
cd "C:\Users\AKASH\OneDrive\Desktop\akash\Industrial-AI-Automobile-Manufacturing-Plant"
$env:PYTHONPATH = "src"
```

```
.venv\Scripts\python -m ddn.demo tag DDN1_BIW_L2_R07_WELDCUR.PV
```

```
.venv\Scripts\python -m ddn.demo vin
```

```
.venv\Scripts\python -m ddn.graph --decisions
```

Window B and C (the two screens)

```
.venv\Scripts\streamlit run apps\ask_app.py --server.port 8501
```

```
.venv\Scripts\streamlit run apps\review_app.py --server.port 8502
```

Neo4j: the signal as a node (Scene 3)

```
MATCH p1 = (t:TimeSeries {tag: 'DDN1_BIW_L2_R07_WELDCUR.PV'})
           <-[:hasTimeSeries]-(a:Asset)
MATCH p2 = (a)-[:partOf*1..5]->(:Site)
OPTIONAL MATCH p3 = (t)-[:hasEvent]->(:AnomalyEvent)
OPTIONAL MATCH p4 = (a)-[:hasEvent]->(:AnomalyEvent)
RETURN p1, p2, p3, p4
```

Neo4j: the car as a picture (Scene 4b)

```
MATCH p1 = (v:Vehicle {vin: 'MDDA5XK00D003924'})-[:hasDefect]->(:Defect)
            -[:causedBy]->(e:AnomalyEvent)<-[:hasEvent]-(:Asset)
MATCH p2 = (v)-[:containsPart]->(:Part {part_no: 'DOOR_SEAL'})
           -[:fromLot]->(:SupplierLot)
MATCH p3 = (v)-[:exposedTo]->(e)
RETURN p1, p2, p3
```

Neo4j: review loop before and after (Scene 6)

```
MATCH (a:Asset)-[r:hasWorkOrder]->(w:WorkOrder {wo_number: '4102964'})
RETURN a.fl AS asset, r.method AS method, r.confidence AS confidence,
       r.status AS status, r.evidence AS evidence
```

Questions for Ask the plant (Scene 5)

```
Which robot caused the cold welds on 7 and 8 September, and which work order fixed it?
```

```
Who was the shift supervisor on 9 September?
```

If Neo4j is not running

```
docker compose up -d
```

If something goes wrong

Problem	Fix
“Can’t connect to Neo4j”	Start Docker Desktop, then `docker compose up -d`, wait 1 minute
“No module named ddn”	In window A, run `$env:PYTHONPATH = "src"` again

Problem	Fix
Ask screen shows an error or an old answer	Ctrl + C in window B, start it again
Ask screen says “Mode: offline”	The OpenAI key is not set in window B: open a new PowerShell and start it again
Review screen has no “Backlog” tab	It was started before `git pull`: Ctrl + C in window C, start it again
You make a mistake while speaking	Pause, then say the sentence again, and cut it later. Or record the scene again